PalletDiT: Code Guide

Manuj Naman, with Claude (Anthropic) as implementation assistant
Anantak Robotics
2026-10-20

How the pallet/ package works, file by file and tensor by tensor. Companion to PALLET_POINTDIT_PAPER.md (the what and why) and PALLET_POSE_HANDOVER.md (the running log). Everything lives on branch pallet-pose; upstream PointDiT files are untouched.

I. Map of the package

file	role
`pallet/targets.py`	the 7 channels, fixed encoding, padding, pixel rays, reconstruction from (range, pose)
`pallet/dataset.py`	`PalletDataset` over the manifest; masks; splits
`pallet/compact.py`	full ray tensors -> compact records (cls, range, pose) with verification
`pallet/manifest.py`	manifest (one row per image) from the training store
`pallet/model.py`	`PalletNet`: PointDiT with 6 in / 6+4 out, loss, generate, checkpoint loading
`pallet/pose.py`	rotation by Kabsch from (theta, phi), camera-centre votes
`pallet/metrics.py`	per-image metrics, aggregation by face distance
`pallet/train.py`	training loop (bf16, EMA, cosine, validation, panels, resume)
`pallet/evaluate.py`	evaluate a checkpoint: 1/3/5 steps, `--hybrid k`, `--rot-from obj`, `--split-from <run>` for a fixed split
`pallet/visualize.py`	inspection panels in the pipeline's colour scheme
`pallet/plot_runs.py`	fit-statistics chart from the logs
`pallet/sequence.py`	sequential estimation: prior rendering from the previous frame, pose propagation/fusion, warm-start evaluation
`pallet/yawfit.py`	per-pick yaw fit experiments (both objectives are blind to a global yaw; kept as the negative result)
`pallet/make_split.py`	split.csv for THIS machine's manifest from a pick list (same picks on laptop and ml-02)
`pallet/scripts/`	data feeders (ray loop, prefetch, puller, compactor), day/night chains, `md2pdf.js`
`pallet/tests/`	9 tests: round trips, geometry identities, pose recovery, GPU smoke test

Run anything with the repo venv: `.venv/bin/python -m pallet.<module>`; tests: `.venv/bin/python -m pytest pallet/tests -q`.

II. Data flow with shapes

```
raw PNG 424x240 (landscape)
  -> rot90 -> upright 424 (H) x 240 (W)
  -> pad 4 rows top + 4 bottom -> 432 x 240

ray record (npz, full or compact)
```

```
  -> targets.encode / targets.reconstruct
  -> cont [6,424,240], valid [6,424,240], cls [424,240]
  -> pad -> [6,432,240] / [432,240]

PalletDataset.__getitem__ returns
  image        [3,432,240] f32 in [0,1]
  target       [6,432,240] f32 (0 where invalid)
  valid_mask   [6,432,240] bool
  cls          [432,240]   int64 (255 = padding)
  pixel_mask   [432,240]   bool
  intrinsics   [4] fx, fy, cx, cy of the padded image
  pallet_P_cam [12] R row-major then p
  cam_in_pallet [3];  face_dist_m, key, scan

PalletNet.forward(batch)   -> loss
PalletNet.generate(image)  -> x_pred [B,6,H,W], logits [B,4,H,W]
targets.decode(x_pred, m)  -> P [3,H,W], range, theta, phi (m, rad)
pose.estimate_pose(...)    -> R, c (camera centre in the pallet frame)
```

Why the image is 432 tall: the ViT patch is 16 px and 424 = 26.5 patches. Padding on the height keeps the width (240 = 15 patches) untouched; `targets.PAD_TOP = PAD_BOTTOM = 4`, `pad_intrinsics` adds 4 to cy. Positional embeddings are interpolated from the pretraining 16x16 grid to 27x15 by PointDiT itself (`model.py: interpolate_pos_encoding`).

III. The targets (`targets.py`)

Channel order of the continuous tensor is `CHANNELS = (px, py, pz, range, theta, phi)`; `HIT_CHANNELS = slice(0, 4)` and `RAY_CHANNELS = slice(4, 6)` name the two validity groups. Classes: `CLS_PALLET=0, CLS_LOAD=1, CLS_GROUND=2, CLS_BG=3, CLS_IGNORE=255`.

Encoding (`encode`): stack P, range, theta, phi; `valid = isfinite`; replace NaN by 0; apply the fixed affine `enc = (raw - TARGET_OFFSET) / TARGET_SCALE` with

`TARGET_OFFSET = (0, 0, 0, 0, π/2, π/2)` `TARGET_SCALE = (2, 2, 4, 4, 0.5, 0.5)`

The scales are roughly the per-channel standard deviations over hit pixels within 12 m (px 1.65, py 1.8, pz 4.5, range 4.5 m; angles 0.3 rad around π/2), chosen once on 2026-10-15 so the network sees O(1) values with noise of scale 1. `decode` inverts it and restores NaN where the mask is false. **There is no per-frame centring or scaling anywhere.**

Geometry helpers. `pixel_rays(fx, fy, cx, cy, H, W)` returns the unit ray of every pixel centre ((u+0.5-cx)/fx, (v+0.5-cy)/fy, 1) normalised, camera x right, y down, z forward. `targets_from_pose(rays, range, pallet_P_cam)` applies

`d = R d_cam,  P = p + range * d,  theta = arccos(d_y),  phi = atan2(d_z, d_x)`

and is the single source of truth used by the tests, the pose voting and the compact reconstruction. `reconstruct(cls, range, pose, intrinsics)` builds the full target dict from a compact record; P equals the pipeline's P to < 1 mm on pallet/load/ground pixels (verified in `tests/test_compact.py`); background pixels become on-ray points instead of splatted lidar points.

IV. Masking, in detail (`dataset.py`, `model.py`)

There are four kinds of "invalid" and each is handled explicitly:

1. **No hit / unknown range** (NaN in the pipeline output, e.g. sky through a dock door): `valid_mask[0:4] = False`, target 0. theta/phi stay valid: they are ray geometry, defined for every pixel.
2. **Too far** (`--max_range_m 12`, default): P/range validity is cleared for hit pixels beyond 12 m (13 % of hit pixels, almost all far ground and background). Their class is still supervised.
3. **Padding rows**: `pixel_mask = False`, all `valid_mask = False`, `cls = 255`, image black.
4. **Background P** is kept as a target (it is a real, if slightly off-ray, 3D point) but never used for pose voting.

In the loss (`PalletNet.forward`):

```
e = randn_like(x) * noise_scale
z = t * torch.where(valid, x, e) + (1 - t) * e      # invalid entries are pure noise at
se = (x_pred - x)**2 * channel_weights / (1 - t).clamp_min(t_eps)**2
flow = se[valid].mean()                              # masked per channel, not per pixel
ce = F.cross_entropy(logits, cls, ignore_index=255)
```

Upstream PointDiT uses one `[B,H,W]` mask repeated over 3 channels (`denoiser.py:115-142`); ours is `[B,6,H,W]` so that angles can be supervised where P cannot. In the metrics, pose votes use `cls_pred ∈ {pallet, load}` on real pixels; the rotation uses all real pixels.

V. The dataset (`dataset.py`)

- `load_manifest(path, picks=None, split=None)` reads `images.parquet`, keeps rows with a tensor, drops `not_a_pick`, and **re-checks that image and tensor exist on disk** (the raw image folders on the laptop are a rolling window).
- `assign_splits(df, val_frac, seed)` splits **by pick**, stratified by `kind_id` (unknown kind is its own group) and topped up at random to `round(val_frac * n_picks)` validation picks. The split is saved as `split.csv` in each run directory.
- `PalletDataset(manifest_or_df, picks, split, crop, color_jitter, max_range_m, pad)`: `load_raw(i)` opens the PNG, rotates it upright (`rot_code 2` = `np.rot90(raw, 1)`), loads the npz (full or compact; a compact record has no `P` key and is reconstructed), reads `cameras.json` of the pick for the intrinsics (cached per pick). `__getitem__` encodes, applies `max_range_m`, pads, optionally crops (crop must be a multiple of 16; cx/cy are shifted exactly), converts to tensors. No flip, no normalisation.
- `decode_sample(sample)` un-pads and decodes back to the npz layout, used by the round-trip test and by the panels.

VI. The model (`model.py`)

```
self.net = PointDiT_models["PointDiT-B/16"](input_size=256, in_channels=6, feature_embe
self.net.out_channels = 6 + 4
self.net.final_layer.linear = nn.Linear(hidden, 16*16*10)   # zero-initialised, as upst
```

That is the whole architectural change. `unpatchify` reads `self.net.out_channels`, so the forward returns `[B,10,H,W]`; channels `0:6` are the x-prediction, `6:10` the class logits. `load_pretrained(path)` loads the released `model_ema1` (or a `pallet/train.py` checkpoint's `ema`), drops keys whose shapes differ — exactly `net.x_embedder.proj1.weight` (`[128,3,16,16]` vs `[128,6,16,16]`) and `net.final_layer.linear.`

{weight,bias} — and asserts nothing else is missing except DINOv3 (net.y_embedder.*, loaded from its own weight file by torch.hub via DINOV3_REPO/DINOV3_WEIGHTS_DIR). trainable_parameters() excludes net.y_embedder (frozen DINOv3).

Image path: dataset gives [0,1]; forward/generate map to [-1,1] (image*2-1) as upstream engine.py:97; PointDiT itself re-maps to ImageNet mean/std before DINOv3 (model.py:575). Mixed precision: torch.autocast(bfloat16) around forward and generate; the loss is computed in float32.

VII. Training (`train.py`)

- **Timestep**: `t = sigmoid(N(P_mean=-0.8, P_std=0.8))`, then `t = 0` with probability `--zero_t_ratio` (0.1). `--zero_t_ratio 1` turns the model into a pure regressor trained only at the inference condition; not used so far.
- **Loss** as in section 4; `--cls_weight` (1.0) and `--channel_weights` (ones) are the only knobs.
- **Optimiser**: AdamW, betas (0.9, 0.95) as upstream, weight decay 0, `--lr` absolute (not scaled by batch), linear warmup `--warmup_steps` then cosine to `--min_lr`. Gradient clipping at 1.0. Non-finite loss skips the step.
- **EMA**: a deep copy updated every step with decay `min(--ema_decay, (1+step)/(10+step))` (warmup so early validations are meaningful). Validation and the saved `ema` weights use it; `--val_raw` also validates the raw weights.
- **Validation** every `--eval_every` epochs on the held-out picks (`--val_max_images 600` even subsample): `generate` with `--steps` (1), `metrics.image_metrics` per image, `aggregate` medians per face-distance bin, printed as a table, saved as `val_<epoch>_<ema|raw>.json`, scalars to tensorboard, 6 prediction-vs-GT panels per validation.
- **Checkpoint** `checkpoint-last.pth` = {model, ema, optimizer, step, epoch, args}; `--resume` continues; `--pretrained` starts from another run's EMA (shapes match fully) or from the public B/256.
- **Throughput** on the RTX 3080 Ti laptop: 18–20 img/s at batch 16 (8.6 GB), ~15 img/s at batch 8.

Typical launch (always detached):

```
setsid nohup .venv/bin/python -m pallet.train --out runs/<name> --pretrained runs/<prev
    --batch_size 16 --lr 5e-5 --warmup_steps 300 --epochs 10 --eval_every 2 --val_raw >
```

VIII. Inference, pose and metrics (`model.generate`, `pose.py`, `metrics.py`)

`generate(image, steps)`: cache DINOv3 features once, `z = 0`, Euler steps on the x-prediction `z <- z + (t_next - t) * (x_pred - z) / (1 - t)`; the last step returns `x_pred` directly. With `steps=1` it is one forward pass at t = 0. Returns encoded `x_pred` and the logits of the last step.

`estimate_pose(P, range, theta, phi, rays_cam, vote_mask, ray_mask, R=None)`:

1. `R = kabsch(rays_cam[ray_mask], dirs_from_angles(theta, phi)[ray_mask])` — SVD of the 3x3 cross-covariance, det-corrected.
2. votes `c_i = P_i - range_i * (R d_i)` over `vote_mask` (predicted pallet/load pixels); `c = median(votes)`, `spread = median |votes - c|`.

The identity behind it: P − c = range·d for every on-ray pixel, so each pixel alone determines c given R. On ground-truth targets the test recovers c to < 1 mm and R to < 0.001 deg.

image_metrics reports: class accuracy and per-class IoU on real pixels; median P and range error on GT pallet/load pixels; median theta/phi error; rotation error (Frobenius-based geodesic angle; arccos was ill-conditioned near 0); origin error |c − cam_in_pallet| with predicted mask and rotation; the same with GT mask and GT rotation (an upper bound on what the voting could do with perfect segmentation); vote spread; number of votes. aggregate takes medians per bin (0,2,3,4.5,6,8,12) m.

VIII b. Sequential estimation (sequence.py, model.generate warm start)

PalletNet.generate(image, steps, x_prior, prior_mask, t0, prior_fill) starts the Euler integration at t0 from the prior instead of at 0 from zeros; unknown prior entries are filled with noise (prior_fill="noise", the training distribution; mixed keeps known entries at the zero-noise mean). sequence.render_prior builds the prior for the next frame from the previous frame's pallet/load pixels (3-px z-buffer splat, P unchanged in the pallet frame, range recomputed) plus the visible ground plane; theta/phi come exactly from the propagated pose. run_pick walks a pick in time order with sources pred (own maps + own smoothed pose), pred_gtpose, oracle, gtcur, maintains the fused pose (smooth_pose, weight pose_alpha on the new vote, propagated with the recorded motion) and reports, per frame, the single/warm-start pose and the pose re-voted with the FUSED rotation (fused_* metrics). Training with priors: PalletDataset(prior=PriorConfig(...)) + PalletNet.forward(batch, prior_p).

Metrics for the product (metrics.image_metrics): lateral_err_cm (offset from the pallet centreline, pallet x), depth_err_cm (pallet z), yaw_err_deg (about the vertical), and their fused_* versions; format_table prints them first. Findings so far: lateral error = distance x yaw error (r 0.98); the yaw error is a per-pick label bias (~1.3 deg std).

IX. Data tooling (compact.py, manifest.py, scripts/)

- **Compact records.** compact_pick(src_rays, dst_rays) keeps cls, range, pallet_P_cam, cam_in_pallet, scan, ts_img_us, verifies the reconstruction against the source before writing, and never deletes unless asked (--delete-src, which the user must approve). The store layout is <store>/<robot>/<pick>/{02/color_NNNN.png, rays/cameras.json, rays/cam2_NNNN.npz}.
- **Manifest.** python -m pallet.manifest walks the store's cameras.json files and writes images.parquet with the columns the dataset uses; kinds come from SensorUnit's picks.parquet.
- **Feeders** (all setsid nohup, logs in ~/Anantak/Data/PicksCopies/_logs/): rays_from_ml02.sh builds ray tensors on the laptop GPU from raw episodes on ana-ml-02 (waits while the GPU holds > 2 GB, i.e. during training), prefetch_from_ml02.sh stages raw episodes, pull_ml02_rays.sh / pull_loop.sh copy picks already built on ana-ml-02 (no GPU), compact_loop.sh folds new tensors into the store every 10 minutes. overnight_*.sh / night2_*.sh are the chains used on 2026-10-15/16.

X. Reading the outputs

- runs/<name>.log: one line per 20 steps — loss, flow, class CE, per-channel RMSE in raw units (m, rad), lr, grad norm, img/s, memory; a validation table every eval_every epochs.
- runs/<name>/val_*.json: per-image rows and the aggregate; panels/: GT panel over prediction panel, caption with the origin error; tb/: tensorboard; split.csv: the exact split; args.json.
- runs/fit_stats.png (refreshed by plot_runs --loop 300) and tensorboard on port 6006.
- Panels colour code: pallet/load pixels by pallet-frame position on a fixed metre scale (R = x −0.75..0.75, G = height 0..2, B = depth 0..1.3), range 0–8 m, theta, phi, class masks (pallet orange, load blue, ground green, background-with-range grey).

XI. Pitfalls met so far

- **Rolling raw window.** `~/Anantak/Data/PicksCopies` keeps only the 60 newest raw pick folders per robot; images vanish. The store is the stable copy; the dataset checks file existence.
- **GPU sharing.** The SensorUnit pipeline app is CUDA and runs out of memory next to a training run; building and training alternate.
- **Waiting on process names.** Two multi-hour stalls came from `pgrep -f <pattern>` matching a shell whose command line merely contained the pattern. Wait on files, PIDs, `pgrep -x`, or GPU memory instead.
- **Validation set drift and leakage.** Each run re-splits a larger store, so a later run has *trained on* part of an earlier run's validation picks; cross-evaluating on another run's split is biased. Compare checkpoints only on picks no run ever trained on: `runs/fixed_val/split.csv` marks the 201 such picks (built 2026-10-19 from the union of all `split.csv` train sets), used as `pallet/evaluate.py --split-from runs/fixed_val`.
- **float32 rotations.** The stored `pallet_P_cam` is float32; use the Frobenius-based angle, not arccos of the trace.
- **Priors and zeros.** At t > 0 an encoded 0 means "a point at the pallet origin"; never feed zeros where a prior is unknown.
- **Splatting predictions.** Only splat predicted pallet/load pixels into a prior; predicted background/ground points land in front of the pallet. Draw the ground plane analytically and only where it is visible.
- **Yaw is not observable from the sequence geometry.** Trajectory consistency and vote dispersion are both invariant to a global yaw; the heading must come from the image (or from corrected labels).